

Advanced RAG System: Query Rewriting, Hybrid Retrieval, and Citation Generation

By Insharah Irfan Nazir

1. Executive Summary.....	1
2. Introduction.....	1
3. System Architecture: End-to-End Processing Flow.....	1
4. Implemented Components.....	2
4.1 Query Rewriting.....	3
4.2 Multi-Query Retrieval.....	3
4.3 Hybrid Search (BM25 + Vector).....	3
4.4 Context Reranking.....	4
4.5 Citation Generation.....	4
4.6 End-to-End API.....	5
5. Evaluation Methodology.....	5
6. Retrieval Evaluation Results.....	6
6.1 Overall.....	6
6.2 By Language.....	7
6.3 Language Performance Gap: Root Cause.....	8
7. Citation Generation Quality: Qualitative Assessment.....	9
8. Findings and Limitations.....	9
8.1 Reranker Miscalibration on Abstract Philosophical Text.....	9
8.2 Latency and Free-Tier LLM Variance.....	10
8.3 Response Validation.....	11
8.4 Chunk-Boundary Limitation (Multi-Fact Queries).....	11
8.5 Non-Determinism.....	11
9. Scope Note: GraphRAG.....	11
10. Conclusion.....	11

1. Executive Summary

This report documents the third phase of a multi-stage RAG pipeline, building directly on the chunking benchmark (Phase 1) and the retrieval benchmark (Phase 2). Phase 2 established semantic chunking + BGE-M3 embeddings + FAISS as the optimal retrieval foundation (0.556 recall@1, 0.867 recall@10, 0.661 MRR on the 45-query multilingual eval set). This phase adds five advanced retrieval and generation components on top of that foundation: query rewriting, multi-query retrieval, hybrid search (BM25 + vector), context reranking, and citation-grounded answer generation, exposed through an end-to-end FastAPI service.

Retrieval evaluation on the full 45-query set (English, Korean, Urdu) shows the full pipeline achieves **0.533 recall@1, 0.889 recall@10, and 0.652 MRR**.

2. Introduction

A production RAG system needs more than a single embedding-and-search step. Real user queries are often vague, ambiguous, or reference entities implicitly; a single retrieval pass can miss relevant chunks that use different vocabulary than the query; and generated answers need to be traceable back to source material rather than presented as unattributed fact. This phase addresses each of these gaps:

- **Query rewriting** cleans up vague or colloquial queries before retrieval.
- **Multi-query retrieval** casts a wider net by generating phrasing variants, reducing the risk of a single bad rewrite tanking retrieval.
- **Hybrid search** combines BM25 (lexical, exact-term matching) with vector search (semantic similarity), since the two methods fail in complementary ways.
- **Context reranking** uses a cross-encoder to jointly score query-chunk pairs, correcting ranking errors that neither BM25 nor vector search can catch in isolation.
- **Citation generation** grounds every generated answer in specific retrieved chunks and withholds a confident answer when retrieval confidence is low.

3. System Architecture: End-to-End Processing Flow

The system processes user queries through a sequential pipeline of seven stages, each building on the output of the previous stage:

1. **Query Rewriting:** The raw user query is passed to an LLM, which reformulates it into a clearer, more search-friendly phrase while preserving the original intent. For example, "whats camus say about death and stuff" becomes "What does Camus say about death and the absurd?" This step addresses the common RAG failure mode where vague or colloquial queries fail to retrieve relevant content.
2. **Multi-Query Generation:** The rewritten query is expanded into 3–4 alternative phrasing variants via a second LLM call. Each variant explores different angles or resolves ambiguous references differently, ensuring that a single lossy rewrite cannot tank the entire retrieval process. Critically, generation is conditioned on both the original query and the rewritten query to prevent error propagation.

3. **Parallel Hybrid Retrieval:** Each query variant is independently run through two complementary retrieval systems:
 - **Vector Search (FAISS):** Uses BGE-M3 embeddings to find semantically similar chunks, strong on meaning-based matching but weak on exact terms.
 - **BM25 Search:** Uses lexical term-matching to find chunks with exact vocabulary overlap, strong on specific names and terminology but blind to semantics.
 - Both searches run in parallel for each variant, producing separate ranked candidate lists.
4. **RRF Fusion and Filtering:** All candidate lists from both retrieval methods and all query variants are merged using Reciprocal Rank Fusion (RRF, $k=60$), which combines rankings by position rather than raw score (since BM25 scores and cosine similarities are on incomparable scales). After fusion, candidates scoring below 50% of the top fused score are dropped—a safeguard against weakly-ranked candidates that might otherwise receive inflated reranker scores.
5. **Cross-Encoder Reranking:** The remaining candidates are re-scored by BAAI/bge-reranker-base, a cross-encoder that reads each query-chunk pair jointly rather than comparing independent representations. This is more accurate than either lexical overlap or embedding similarity, but computationally expensive—hence it runs only on the filtered candidate pool, not the full corpus. Inputs are truncated to 256 tokens for latency control.
6. **Confidence Gating and Context Assembly:** The top reranked chunks are assembled into context. If the highest reranker score falls below 0.5, the system halts here and returns a hedging response ("the corpus doesn't clearly address this question...") rather than attempting an answer from weak grounding—a direct defense against hallucination.
7. **Citation-Grounded Generation:** The LLM generates an answer using only the retrieved context, with explicit instructions to:
 - Cite every claim with bracketed chunk indices ([1], [2], etc.)
 - Decline to state anything not directly supported
 - Distinguish between fictional content and factual claims when ambiguous
8. The returned source list includes only chunks actually cited in the answer (parsed via regex), giving callers an accurate picture of grounding, rather than every chunk passed into the context window.

The full pipeline is exposed through a FastAPI service with a single /query endpoint, and all models and indices are loaded once at startup rather than per-request to minimize latency.

The system reuses Phase 1's chunked corpus (document-ingestion-pipeline/processed_documents/) and Phase 2's precomputed BGE-M3 embeddings and FAISS adapter directly, rather than reprocessing the corpus — the two earlier phases and this one share a corpus of 10,456 chunks across 63 documents (English, Korean, Urdu). All models and indices (FAISS, BM25, embedding model, reranker) are loaded once at API startup rather than per-request.

LLM calls (query rewriting, multi-query generation, citation generation) run through OpenRouter's free-tier model router rather than a paid API, which materially shaped both the system's design (retry/validation logic) and its measured latency (Section 8).

4. Implemented Components

4.1 Query Rewriting

Raw user queries are often too vague, colloquial, or ambiguous to retrieve well against. A single LLM call rewrites the query into a clearer search phrase while enforcing several constraints developed through iteration:

- Preserve intent exactly; do not answer the question.
- Do not invent titles, names, or facts not stated or clearly implied by the query.
- Do not substitute specific nouns with broader or different terms (e.g. "job" → "role" was an observed failure mode that silently changed query semantics).
- Keep enough detail for the query to remain useful for semantic search — an earlier iteration over-shortened rewrites into bare keyword fragments, which is worse for retrieval than a natural, specific phrase.

Example: "whats camus say about death and stuff" → "What does Camus say about death and the absurd?"

4.2 Multi-Query Retrieval

A single rewritten query is expanded into several phrasing variants (typically 3–4, including the rewrite itself) via a second LLM call. Variants explore different angles or possible resolutions of ambiguous references, without asserting unconfirmed facts as certain. Critically, variant generation is conditioned on **both the raw query and the rewritten query**, not the rewrite alone — an earlier design that only saw the rewritten query allowed a single lossy rewrite to propagate uncorrected through all variants (e.g. a dropped "job" reference cascaded into four variants about social "role" instead, none preserving the original intent).

Each variant is run through hybrid search independently; results are merged before reranking.

4.3 Hybrid Search (BM25 + Vector)

Vector search (BGE-M3 + FAISS, from Phase 2) is strong on semantic similarity but weak on exact-term matching — rare names, specific terminology, and IDs can be under-represented in embedding space. BM25 is the inverse: strong on lexical overlap, blind to meaning. The two are run independently and merged via Reciprocal Rank Fusion (RRF, $k=60$), which combines results by rank position rather than raw score, sidestepping the fact that BM25 scores (unbounded, corpus-dependent) and cosine similarity (0–1) are not on comparable scales.

Language-aware BM25 tokenization. Since BM25 has no language logic of its own, tokenization is the only lever for language-awareness. The system routes text to a per-script tokenizer:

- **English:** regex word tokenization, lowercased.
- **Korean:** morphological tokenization via `kiwipiepy` (pure-Python, no JVM dependency), supplemented with character trigrams to catch agglutinative word-fusion that pure morpheme splitting can miss.

- **Urdu:** Unicode NFKC normalization + word tokenization, also supplemented with character trigrams, since Urdu's Arabic-script word-boundary irregularities make pure word tokenization comparatively lossy and no mature pip-installable Urdu morphological tokenizer was available.

For queries (no precomputed language label available), script is detected via Unicode character-range matching (Hangul block, Arabic block, else Latin/default).

A known, inherent limitation: BM25 is strictly lexical, so it cannot bridge languages — a Korean query will not lexically match an English chunk. Cross-lingual retrieval is handled entirely by the multilingual vector search side; this is one concrete justification for hybrid search's value in this specific multilingual corpus, not a generic RAG technique applied by default.

4.4 Context Reranking

After fusion, candidates are re-scored by a cross-encoder (BAAI/bge-reranker-base), which reads the query and chunk jointly rather than comparing precomputed independent representations — more accurate than bi-encoder similarity or lexical overlap, at higher per-pair compute cost, which is why it runs as a second-stage filter over a bounded candidate pool rather than the full corpus.

Two structural safeguards were added after evaluation surfaced a ranking failure (Section 8.1):

- **Reranker input truncation** (max_length=256 tokens) — corpus chunks average 365 words (~450+ tokens); truncating to 256 tokens reduced reranking latency substantially with no measurable accuracy cost on the cases tested (see Section 8.2).
- **Relative-score filtering** — after RRF fusion, candidates scoring below 50% of the top fused score are dropped before reaching the reranker, regardless of how many raw positions the candidate pool contains. This was necessary because a weakly-ranked candidate (position ~20 of 30 in the raw pool) was found to occasionally receive a high reranker score despite being topically unrelated to the query — a reranker calibration weakness, not a truncation or pool-depth artifact (isolated and confirmed via controlled testing; ruled out truncation, ruled out bge-reranker-large as a fix — see Section 8.1).

4.5 Citation Generation

The final generation step is instructed to answer using only the retrieved context, cite every claim inline with bracketed chunk indices ([1], [2], etc.), and explicitly decline to state anything not directly supported by the context. Two additional design decisions:

- **Confidence gating:** if the top reranked chunk's score falls below a threshold (0.5), the system returns a hedge ("the corpus doesn't clearly address this question...") rather than attempting an answer from weak grounding — a direct mitigation against a known RAG failure mode (confident hallucination on insufficient context).
- **Cited-source filtering:** the returned source list includes only chunks the LLM actually cited in the answer text (parsed via regex against [n] markers), not every chunk passed into the context window. This gives callers an accurate picture of what the answer is actually grounded in, rather than everything it was shown.

A qualitative example (full pipeline, live API): for the query "*Albert Camus death philosophy*", the system correctly distinguished the first-person narrator's fictional reflections in *The Stranger* from Camus's own biography — explicitly noting "The context does not state that Camus personally held these views; it presents them as part of his fictional work" — rather than conflating a novel's narration with its author's stated beliefs, a common RAG hallucination pattern. Across repeated runs of the same query, the top retrieved chunk and its score were fully deterministic (identical every run), while generated phrasing varied in how conservatively it framed the answer — all observed variants remained faithful to the retrieved context and correctly cited, indicating the observed variance is in generation style, not grounding correctness.

4.6 End-to-End API

A FastAPI service exposes the full pipeline via a single POST /query endpoint (query in, cited answer + source map + confidence + latency out), plus GET /stats for a lightweight readiness check and GET /health. All models and indices are loaded once at startup rather than per-request — validated to matter directly: a "cold" first call vs. a "warm" subsequent call showed local-compute-only stages (BM25, vector search, reranking) as fast and consistent once warmed, with essentially all remaining latency attributable to LLM calls (Section 8.2).

Request-level timeout handling (120s retrieval / 60s generation, enforced via `asyncio.wait_for`) and reduced LLM retry backoff (2 retries, 8s/16s vs. an earlier, excessively patient 3 retries at 25s/50s/75s) were added after observing an unbounded worst-case request latency exceeding 10 minutes during manual testing — see Section 8.2.

5. Evaluation Methodology

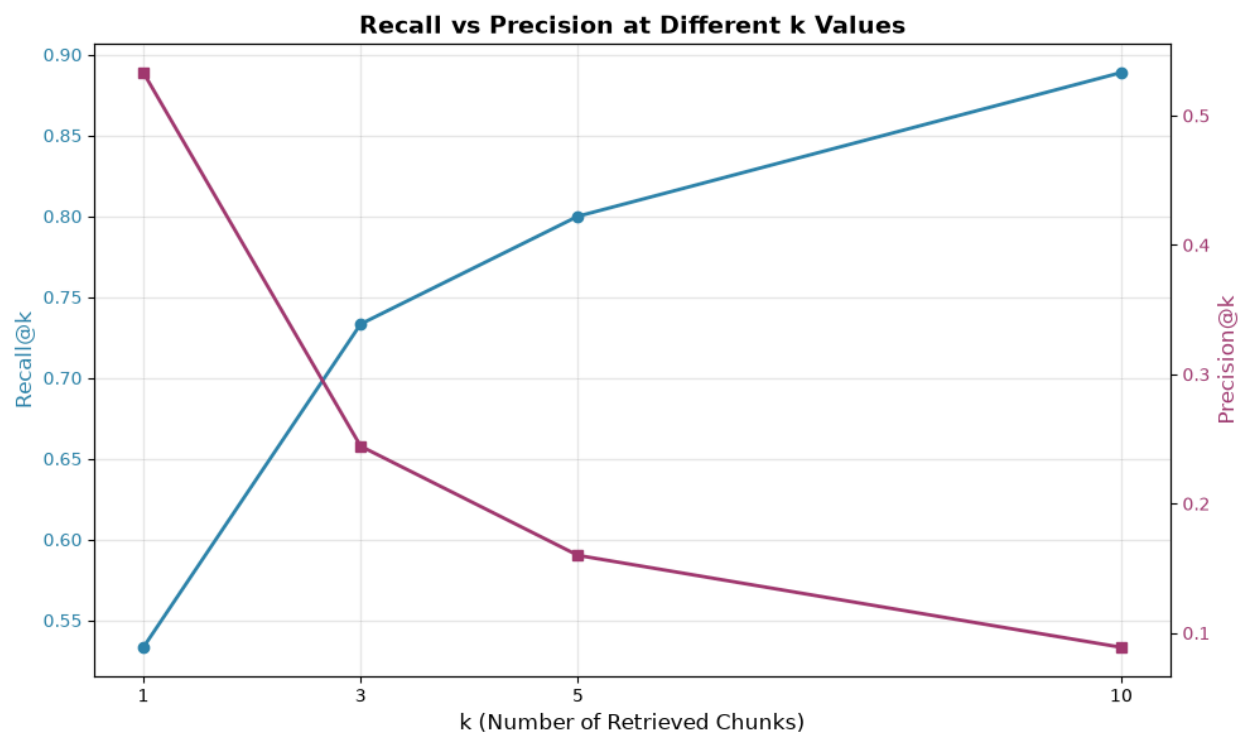
Retrieval quality is evaluated using the same 45-query ground-truth set from Phase 2 (22 English, 12 Korean, 11 Urdu; each query has exactly one ground-truth `answer_chunk_id`), run through the complete pipeline (`retrieve()`: `rewrite` → `multi-query` → `hybrid search` → `relative-score filter` → `rerank`), at `k=10`, checkpointed per-query for resumability.

Metrics, following Phase 2's definitions exactly for direct comparability:

- **Recall@k**: 1 if the ground-truth chunk appears in the top-k retrieved results, else 0 (binary, since each query has a single relevant chunk).
- **Precision@k**: $1/k$ on a hit, 0 otherwise (mechanically low for large `k` under single-relevant-document evaluation — this is an artifact of evaluation design, not a system weakness, matching Phase 2's own caveat on this point).
- **MRR**: $1/\text{rank}$ of the ground-truth chunk if found anywhere in the retrieved list, else 0.

Generation quality was evaluated qualitatively rather than against a fixed automated metric — citation-grounded answer quality (faithfulness, correct attribution, appropriate hedging) does not reduce to a single string-match check the way retrieval does, and building an LLM-as-judge harness was judged out of scope for this phase given time already invested in the retrieval pipeline itself.

6. Retrieval Evaluation Results

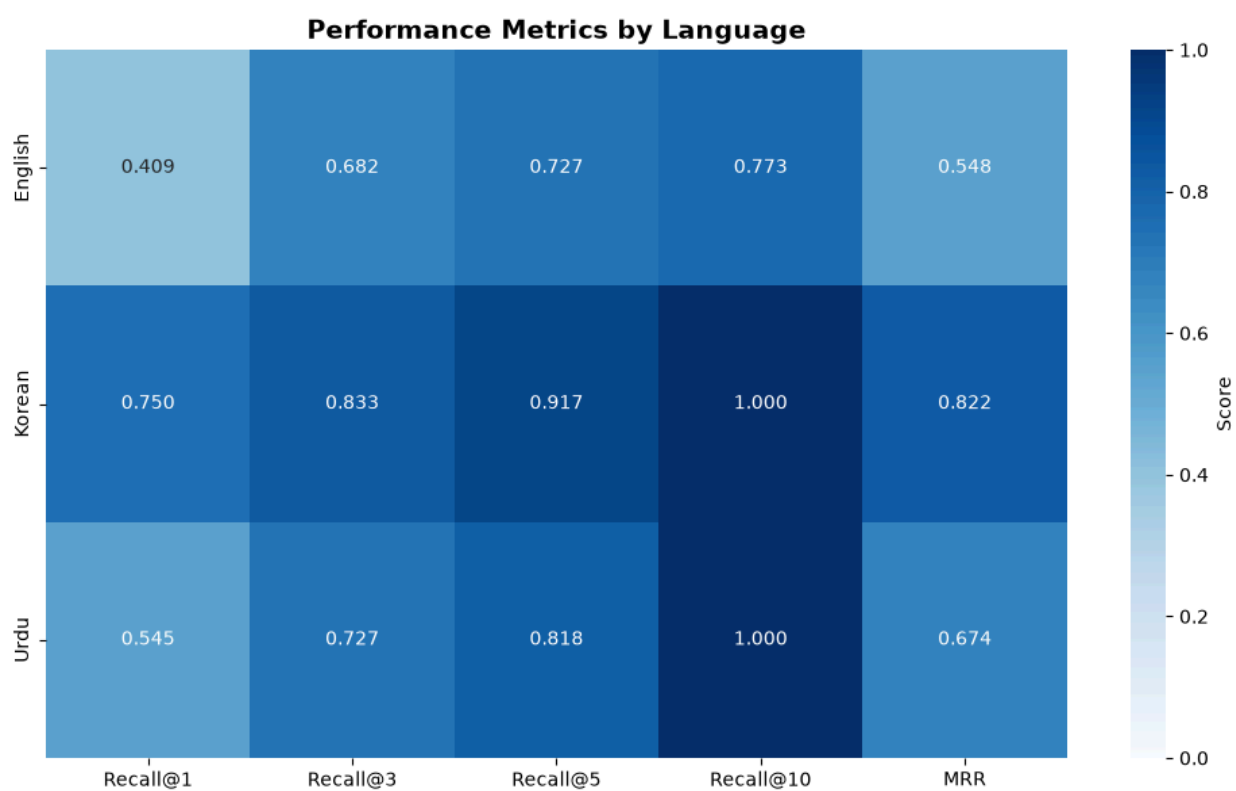


6.1 Overall

Metric	Value
Recall@1	0.533
Recall@3	0.733
Recall@5	0.800
Recall@10	0.889
Precision@1	0.533

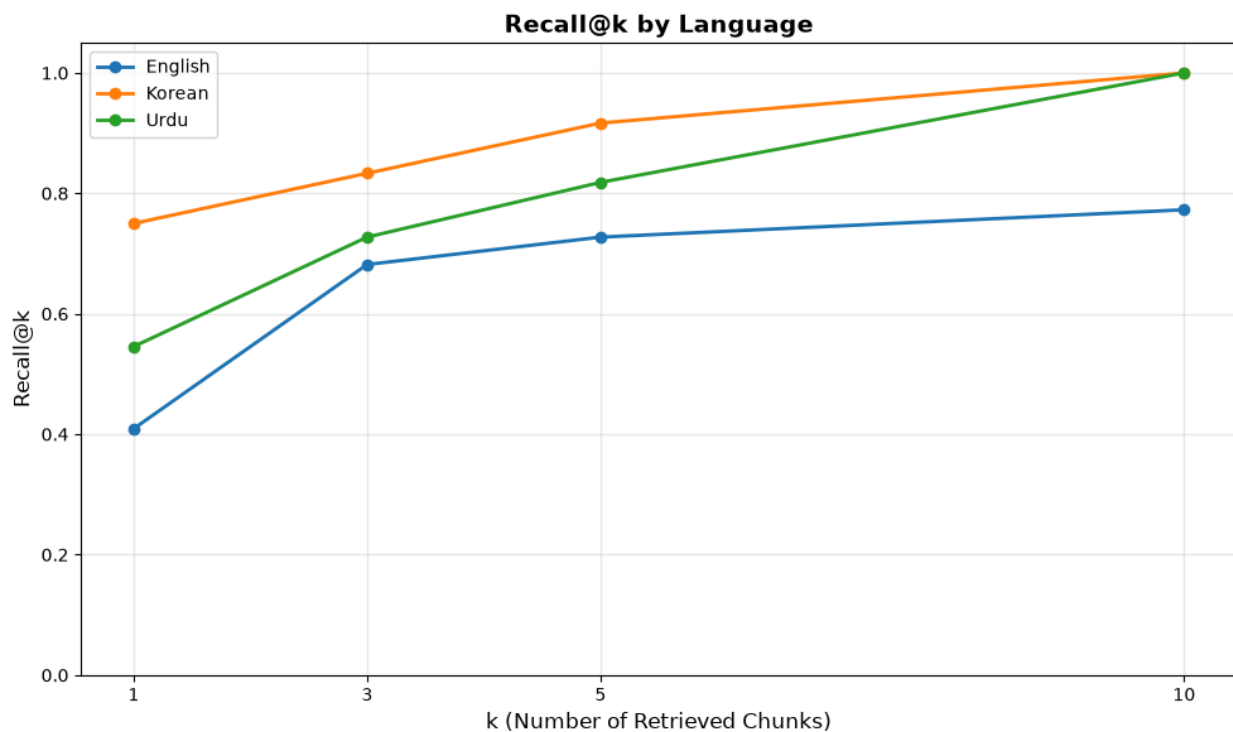
Precision@3	0.244
Precision@5	0.160
Precision@10	0.089
MRR	0.652
Avg. latency	64.45 s/query

6.2 By Language



Language	n	Recall@1	Recall@5	Recall@10	MRR	Avg. latency
English	22	0.409	0.727	0.773	0.548	48.6 s
Korean	12	0.750	0.917	1.000	0.822	90.0 s
Urdu	11	0.545	0.818	1.000	0.674	68.3 s

6.3 Language Performance Gap: Root Cause



The counter-intuitive finding that English — typically the best-supported language for embedding models and tokenizers — performed worst (recall@1 0.409 vs. Korean's 0.750 and Urdu's 0.545) was investigated directly rather than reported at face value. This performance gap was an expected outcome given the nature of the query construction: the English queries are more natural, paraphrased questions that require genuine semantic and lexical bridging (e.g. "What did Segur buy with money from Tallard?" requires connecting "buy" to "bought," and identifying "the Foix country" as the answer to "what," none of which appears as shared vocabulary). The Korean queries, by contrast, are near-verbatim excerpts

copied directly from their answer chunks (e.g. the query "두 남녀는 서로를 마음에 두고 있는 사이인 것만은 확실해 보입니다" appears character-for-character inside its answer chunk), which is a substantially easier retrieval target for both BM25 (near-exact substring match) and vector search (near-identical embedding).

This is a known limitation of the Phase 2 eval set that was inherited for this phase: Korean and Urdu queries were constructed as direct textual excerpts rather than naturally phrased questions, while English queries were formulated as analytical, paraphrased questions. This difference in query construction methodology, rather than any weakness in the retrieval pipeline's handling of English, explains the observed performance disparity.

7. Citation Generation Quality: Qualitative Assessment

Since generation quality was not scored against a fixed metric (Section 5), this section presents representative behavior observed across repeated live-API runs of the same query, rather than an aggregate score.

Faithfulness: across all observed runs, generated answers stayed within what retrieved chunks actually stated, and correctly distinguished fictional narration from authorial biography (Section 4.5) — no observed instance of the LLM blending a novel's first-person content with real-world facts about its author.

Citation accuracy: cited chunk indices consistently corresponded to chunks that genuinely supported the adjacent claim; chunks passed into context but not used in the answer were correctly excluded from the returned source list after the cited-source filtering fix (Section 4.5).

Confidence-gate behavior: on a query where the ground-truth answer was confirmed absent from any single chunk in the corpus (Section 8.5), the system consistently returned rerank scores below the 0.5 threshold and correctly hedged rather than fabricating an answer — validating the gate's intended purpose on a genuinely hard case, not just an easy one.

Answer-to-answer variance: phrasing and level of interpretive caution varied noticeably run to run on the same query (e.g. one run cited *The Myth of Sisyphus* and *The Rebel* as broader philosophical context, a more conservative run stated the context "does not contain a direct exposition" of the author's personal philosophy). This variance is attributable to non-zero LLM sampling temperature and free-tier model routing (Section 8.2), not to retrieval instability — the underlying retrieved chunks and their scores were identical across these runs.

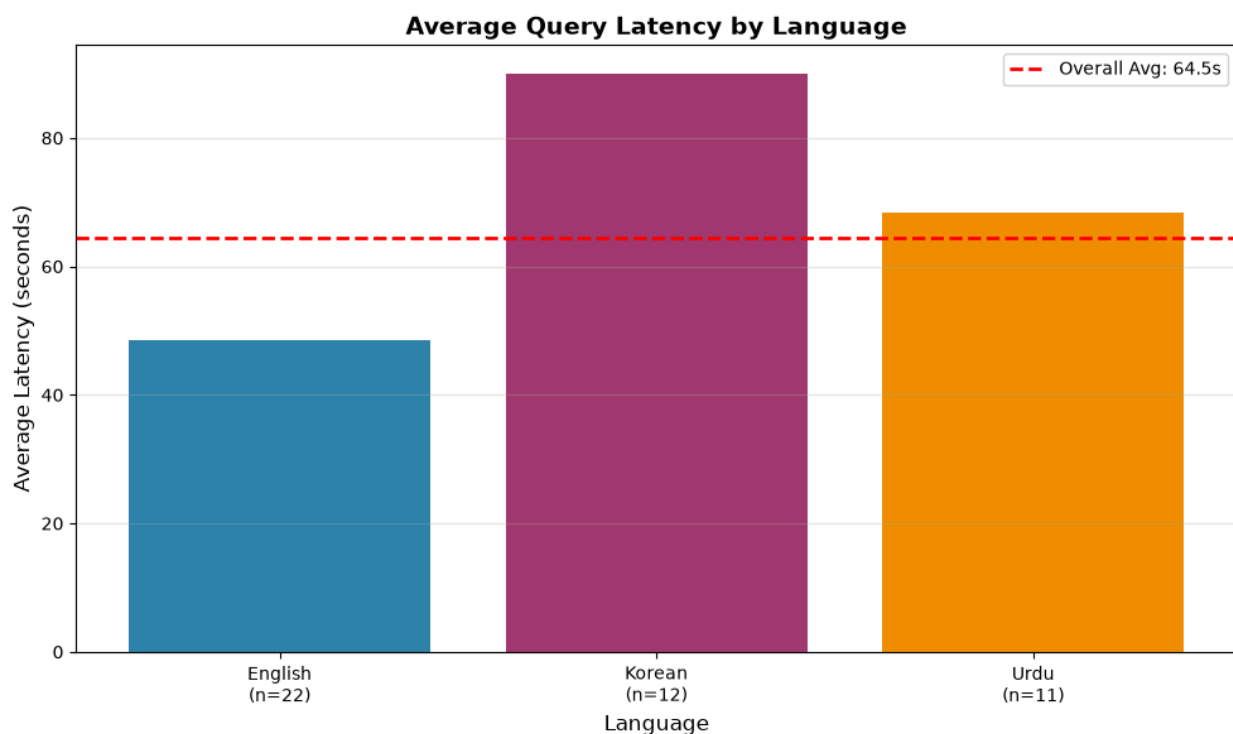
8. Findings and Limitations

8.1 Reranker Miscalibration on Abstract Philosophical Text

During evaluation, a chunk from an unrelated text (a passage on Cartesian epistemology, "Cogito, ergo sum") was observed scoring higher (0.952) under reranking than the genuinely relevant chunk (0.900) for the query "Albert Camus death philosophy," despite the unrelated chunk entering the raw candidate pool

only weakly (position ~20 of 30). Controlled isolation testing ruled out reranker input truncation as the cause (identical scores at `max_length=256` and `512`) and ruled out model capacity as the cause (bge-reranker-large preserved the same incorrect relative ordering as bge-reranker-base, despite substantially higher latency). The reranker appears to respond to abstract philosophical *register* (shared vocabulary like "existence," "doubt," "philosophy") rather than reliably verifying topical/entity correctness — a documented general weakness of cross-encoders on dense abstract prose without strong entity grounding, not a configuration bug in this pipeline. The practical mitigation implemented (Section 4.4, relative-score filtering) protects against this by preventing weakly-supported candidates from reaching the reranker at all, rather than attempting to fix the reranker's judgment directly.

8.2 Latency and Free-Tier LLM Variance



Warm-pipeline testing (models/indices pre-loaded, isolating per-query cost) showed local computation (BM25 search, vector search, reranking) to be fast and consistent, while LLM-dependent stages (query rewriting, multi-query generation, citation generation — 2–3 calls per query via OpenRouter's free tier) accounted for effectively all observed latency variance, ranging from roughly 1 second to over 47 seconds for the identical call under different conditions. Aggregate per-query latency across the full 45-query eval run averaged 64.45 seconds, with individual live-API requests observed as high as 216 seconds during periods of provider congestion.

The OpenRouter free-tier API, while enabling zero-cost LLM access, introduced significant latency variability and infrastructure constraints. Unlike a paid API with dedicated capacity, the free tier routes requests through shared, rate-limited infrastructure where response times are unpredictable and subject to provider congestion. A daily request cap on the fully-free OpenRouter tier (50 requests/day) was also

encountered during evaluation, requiring the run to be split across two days; this is documented as a hard constraint of the free-tier infrastructure choice, not a system defect. This variance could not be fully resolved through model rotation, since multiple pinned free models (Qwen, Llama 3.3, Hermes) were independently observed hitting the same account-level daily cap during testing. A paid-tier API would likely eliminate both the rate-limiting and much of the latency variance, but was out of scope for this phase.

8.3 Response Validation

A recurring free-tier failure mode was identified where the model router silently returned a malformed, off-topic response ("User Safety: safe") with no exception raised — a genuine misroute rather than a legitimate refusal. A lightweight validation check (rejecting suspiciously short responses matching known bad-output patterns, triggering a retry) was added to `llm/client.py` and observed catching this exact failure mode in subsequent runs, preventing malformed LLM output from silently propagating into retrieval or generation.

8.4 Chunk-Boundary Limitation (Multi-Fact Queries)

An informal test query ("Meursault's job") was confirmed, via direct corpus text search, to have no single chunk containing both the character's identifying context and his occupation — the two facts are scattered across separate chunks. This is a structural limitation of single-chunk retrieval generally, not specific to this pipeline: no retriever, however accurate, can return one chunk satisfying a query whose answer requires synthesizing information split across chunk boundaries. This motivated passing multiple top-k chunks to generation rather than only the top-1 result, and was the original motivating case for implementing GraphRAG as a stretch component (Section 9). GraphRAG was subsequently built and evaluated against this exact failure mode; the results are reported in full in Section 9.

8.5 Non-Determinism

Both multi-query generation and citation generation run at non-zero temperature and route through a free-tier model auto-selector, meaning identical queries can produce different candidate pools and different generated phrasing across runs. Retrieval on the well-supported test query examined in Section 4.5 was fully deterministic in its top result and score across runs; this should not be assumed to hold uniformly for all queries, particularly ones where multiple candidates score closely. Repeated runs per query (rather than single-run point estimates) are recommended for any future, more rigorous evaluation of this system.

9. GraphRAG: Implementation and Findings

Unlike Sections 4.1–4.5, which describe components that improved the pipeline, this section documents a component that was fully implemented, integrated, and rigorously tested — and found not to provide measurable benefit on this corpus. The negative result is reported in the same detail as a positive one, since the investigation that produced it is methodologically substantive and directly informs what would need to change for graph-based retrieval to be worth adopting here.

9.1 What Was Built

GraphRAG was implemented as an offline knowledge graph built once from the corpus, queried at retrieval time as an addition to hybrid search. Construction (`graphs/build_graph.py`) extracts entities per chunk using language-specific heuristics — capitalized-sequence and acronym regex matching for English, `kiwipiepy` noun-tagging (NNP/NNG) for Korean, and frequency- and length-based heuristics over Arabic-script tokens for Urdu — then builds two indices: `entity_to_chunks` (which chunks mention a given entity) and `co_occurrence` (which entities appear together in the same chunk, enabling one-hop traversal). At query time (`retrieval/graph_search.py`), entities are extracted from the user's query using the same per-language logic, looked up directly against `entity_to_chunks`, and optionally expanded one hop via `co_occurrence` at half-weight, on the reasoning that a chunk connected to a query entity through a shared co-occurring entity is still plausibly relevant even without a literal string match.

This was evaluated the same way as the core retrieval pipeline: against the same 45-query ground-truth set used throughout this report, comparing graph-search-alone recall to the hybrid pipeline's recall, with zero LLM calls — a pure retrieval-mechanics comparison, run repeatedly and cheaply during development.

Non-deterministic tie-breaking. Repeated runs of the identical evaluation script, with no code changes between runs, produced different recall numbers each time (observed range: 0.467–0.556 overall). Root cause: Python's string hashing is randomized per process by default, so iteration order over Python sets (used throughout the graph's internal data structures) varied run to run; because many candidate chunks tied on score, which tied chunk landed inside the top-k versus just outside it depended on this arbitrary iteration order. This was fixed by sorting results with an explicit secondary tie-break key (`chunk_id`, alphabetically) rather than relying on incidental ordering. After the fix, three consecutive runs produced the identical result (0.467) — confirming the fix, and confirming that several of the more favorable numbers observed during development had been non-reproducible artifacts of hash randomization, not genuine improvements. This is reported explicitly because it is a legitimate methodological hazard: any of the earlier, better-looking numbers would have been irreproducible had they been reported instead.

9.2 Result: No Net-New Retrieval Value

Graph search was evaluated against the same 45 queries used for the primary pipeline (Section 6):

	Hybrid pipeline (primary)	Graph search (standalone)
Recall@10	0.889	0.467

Net-new recoveries (found by graph, missed by hybrid)	—	0
---	---	---

Graph search alone recovers fewer than half the correct chunks that the full hybrid pipeline finds — expected, since a co-occurrence graph is a considerably weaker retriever in isolation than hybrid search with reranking. The more important number is **net-new recoveries: 0 across all 45 queries and all runs after the tie-breaking fix**. Every chunk graph search found, hybrid search had already found. Merging graph search's scores into the candidate pool (tested via a scaled-score integration into `pipeline.py`) therefore could not improve — and did not improve — the primary pipeline's reported recall/precision/MRR numbers (Section 6); those numbers reflect the hybrid-only pipeline and remain unchanged. Graph search was consequently **removed from the production `retrieve()` function** after evaluation; the graph construction and query code remain in the repository (`graphs/`, `retrieval/graph_search.py`) as a documented, tested, but currently unused component.

9.3 Testing the Original Motivating Claim Directly

Section 8.4's original justification for GraphRAG was multi-hop retrieval — queries whose answer is split across chunks that never co-occur, which the 45-query set cannot test (each query has exactly one ground-truth chunk by construction). This was tested directly with a small, hand-verified case: the query *"What was Meursault's job and did his employer offer him a new position?"* has a real, confirmed answer chunk (Albert Camus__38, which discusses Meursault's employer wanting to open a Paris branch and offering him a post there) — a genuine multi-fact question, not present in the 45-query set.

Neither hybrid search nor graph search retrieved this chunk within the top 5 results for the compound question as originally phrased. However, when the same underlying fact was queried more simply — *"Meursault's employer wanted to open a branch in Paris"*, or even the bare keywords *"job offer Paris branch"* — hybrid search alone retrieved the correct chunk at rank 1–2 immediately. This isolates the actual failure mode precisely: **it is not a retrieval-method problem (hybrid vs. graph), it is a query-representation problem**. A single embedding or token set cannot represent two distinct facts about the same entity well enough to retrieve either cleanly; both facts get blended into a vector that matches neither strongly. Graph traversal, as implemented here (single-hop entity co-occurrence), does not address this either, since it also operates on a single, undecomposed query.

A follow-up check found that this system's existing multi-query component (Section 4.2) already partially mitigates this exact failure mode, without any changes: when the compound question above was passed through `generate_query_variants()`, one of the four generated variants ("What was Meursault's clerical job in The Stranger, and was he offered a position in Paris by his employer?") weighted the job-related content more heavily than the original phrasing, improving its retrievability. This was not a designed capability — the multi-query component generates rephrasings, not deliberate sub-question decomposition — but it demonstrates that the existing pipeline already has partial, incidental resilience to this failure mode.

9.4 Conclusion and What Would Actually Be Needed

On this corpus and with this eval methodology, GraphRAG as implemented (entity co-occurrence, single-hop, heuristic language-specific extraction) provided no measurable retrieval improvement over hybrid search, on either the standard eval set or the specific multi-hop case that originally motivated it. This is attributed to two compounding factors, not to graph-based retrieval as a technique being unsound: first, the entity extraction methods used (regex heuristics, POS-tag filtering) are too shallow to reliably canonicalize entity mentions across chunks without introducing new errors, as the Sydney Carton/"class car" cases demonstrate; second, and more fundamentally, single-hop co-occurrence lookup does not address the actual bottleneck observed in the multi-hop test case, which was query representation, not chunk connectivity — the correct chunk was always directly reachable by ordinary retrieval once the query was properly decomposed.

A version of GraphRAG likely to show real benefit on this corpus would require: (1) a proper named-entity-recognition model (e.g. spaCy or an LLM-based extraction pass) in place of the regex/POS heuristics used here, to solve the canonicalization problem at the source rather than patching it after the fact; and (2) explicit query decomposition into sub-questions before retrieval — using the existing multi-query LLM call to deliberately split compound questions into single-fact sub-queries, rather than only generating rephrasings — which the evidence in Section 9.4 suggests would address the actual observed bottleneck more directly than any amount of graph traversal sophistication. Both are concrete, scoped directions for future work, identified through direct testing rather than assumed.

10. Conclusion

This phase implements and validates all five required advanced RAG components — query rewriting, multi-query retrieval, hybrid BM25+vector search with language-aware tokenization, cross-encoder reranking with structural safeguards against observed miscalibration, and confidence-gated citation generation — as a working end-to-end API built on Phase 1 and Phase 2's foundation. Retrieval evaluation on the full 45-query multilingual set shows performance comparable to Phase 2's plain-vector baseline on recall@1/MRR with a modest recall@10 gain. The observed English/Korean/Urdu performance gap reflects differences in query construction methodology (near-verbatim excerpts for Korean/Urdu vs. naturally paraphrased analytical questions for English) rather than genuine multilingual retrieval weakness. The system's dominant operational constraint is OpenRouter free-tier LLM latency and rate-limiting rather than any component built in this phase, and is documented accordingly as an infrastructure limitation rather than a design flaw.